Qualcomm

Support RT tasks with sched-ext

Tengfei Fan and Aiqun Yu

tengfan@qti.qualcomm.com

aiquny@qti.qualcomm.com

Co-Contributor:

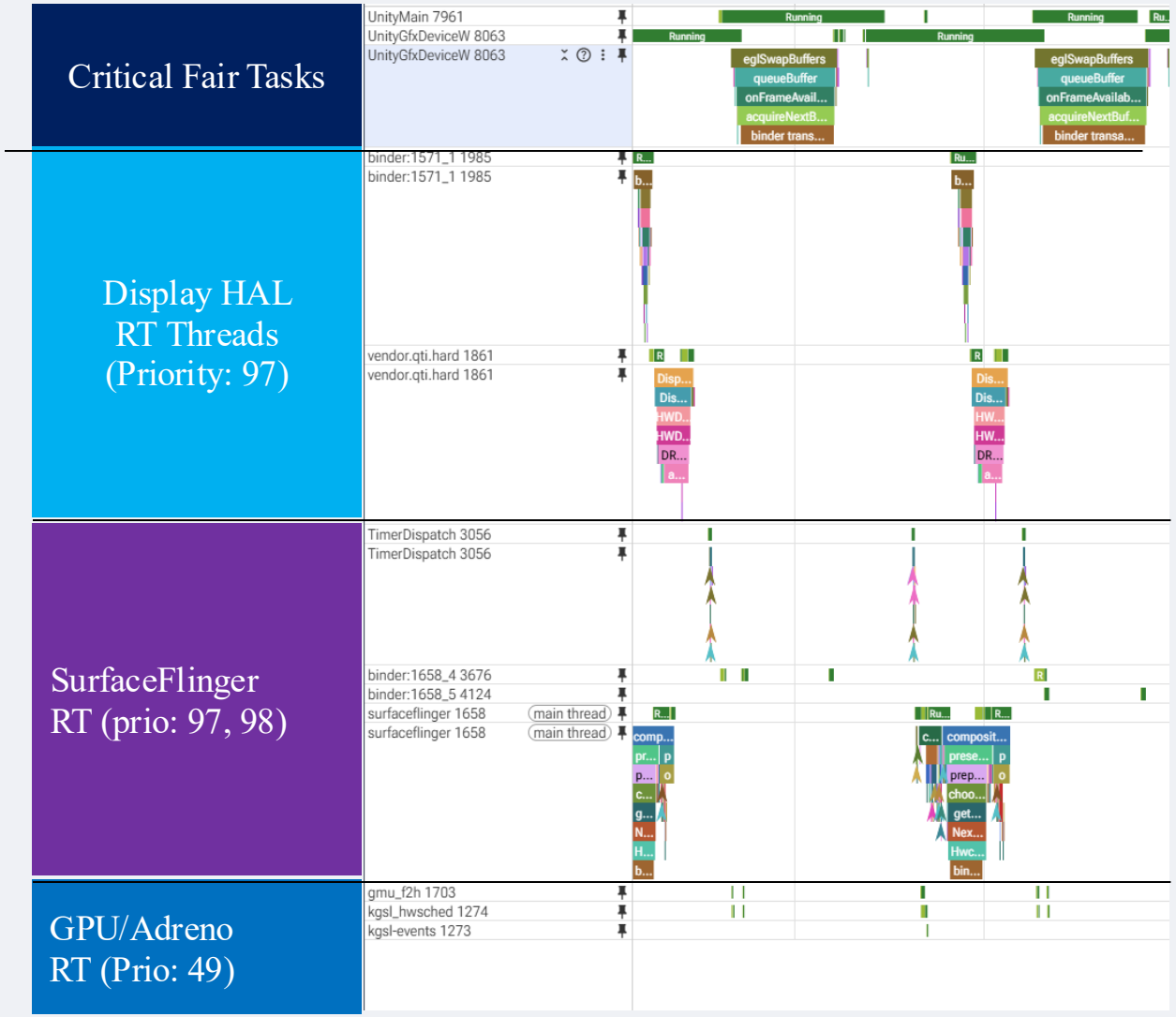
Yucong Zhang and Rohan Tabish



Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries.
Qualcomm patents are licensed by Qualcomm Incorporated.

RT Tasks play an important role in today's Android gaming, livestream, and video scenarios

Critical Fair tasks and RT tasks jointly render a frame



RT task count and workload in different scenarios

scenarios	FPS	RT / total task(percentage)	RT workload in cluster (average)
Honor of King	120	53 / 443 (11.96 %)	24% ~ 57%(45.5%)
Game For Peace	165	29/202 (14.36%)	41% ~ 56%(49.2%)
	120	28/220(12.7)	30% ~ 49%(45.8%)
Honkai: Star Rail	60	26/158 (16.46%)	16% ~ 51%(42%)
Tencent Video	60	41/ 771(5.29%)	13% ~ 42%(28.2%)
BiliBili Video	30	49 / 545 (8.99 %)	10% ~ 41%(27.1%)
TikTok livestream	30	29 / 254 (11.42 %)	10% ~ 20%(13.9%)

In real-world application scenarios, RT tasks:

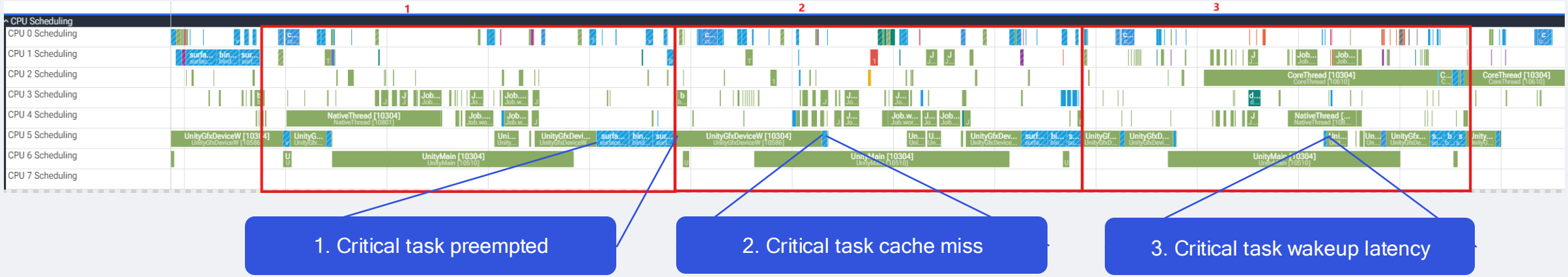
- RT task's count is not small
- RT is 5–16% of all tasks and 14–50% of cluster load
- The higher the FPS, the heavier the RT workload percentage

Displaying a single frame involves RT tasks from multiple processes, directly affecting visual quality and user experience.

Critical tasks can be interfered with by RT task, potentially causing frame drops in the gaming scenario

TODAY

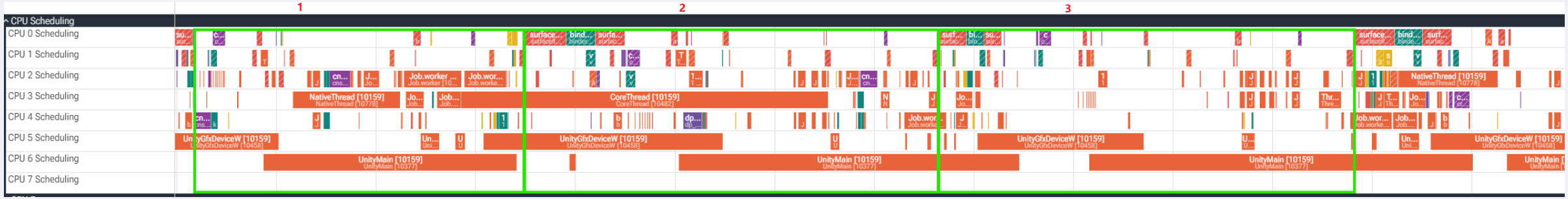
- Critical tasks (UnityMain & UnityGfxDeviceW) can be interfered with by frame-related RT
  Data captured from 120FPS Honor of King



Consistent interference from SurfaceFlinger and binder RT tasks on the critical threads across multiple frames

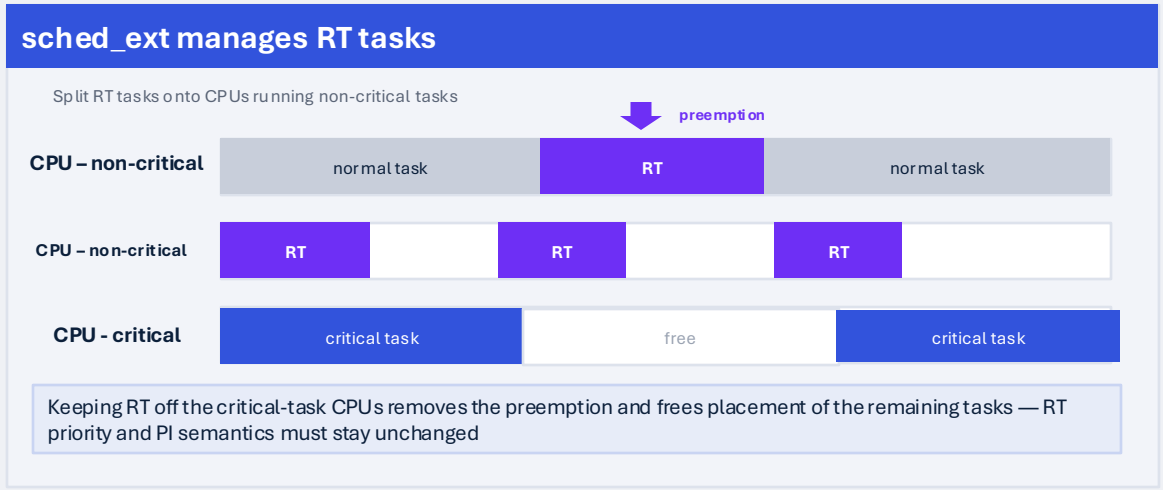
EXPECTATION

- **Requirement: Critical game tasks can run without RT interference**



How to isolate critical tasks from the RT task interference with sched-ext

Option 1 Extend sched-ext aware of RT task

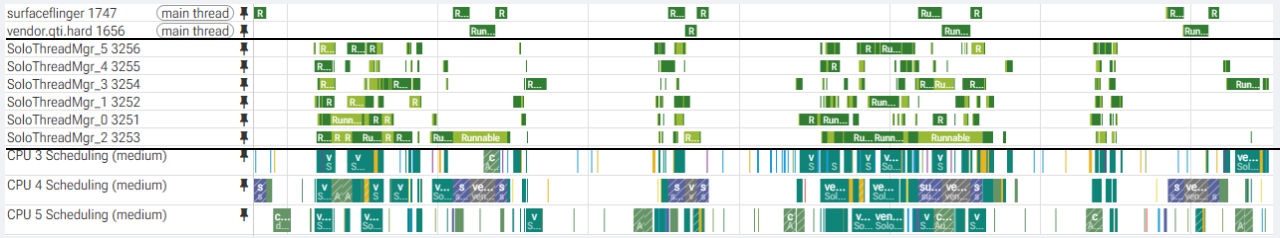


Option 2 Extend RT scheduler aware of the non-critical cpus



RT tasks need to be load-balanced together with FAIR tasks

TikTok livestream scenario



- 2 RT tasks run periodically in the same frame.
- 6 FAIR tasks run periodically in the same frame.
- Unnecessary runnable time is observed.

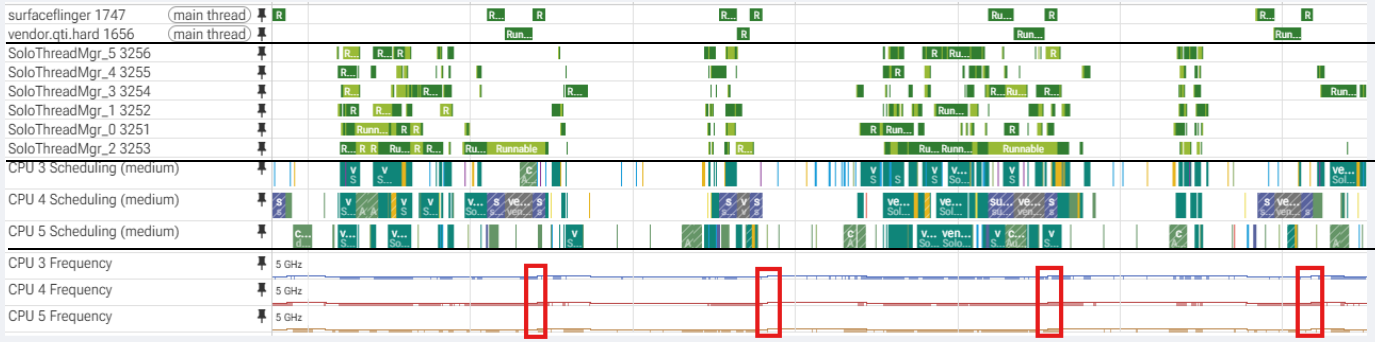
Thread Name	Count COUNT 353	CPU Time ↓ SUM 81.34ms	CPU Time % SUM 100.00%
AudioRecord	38	9.670ms	11.89%
surfaceflinger	9	7.868ms	9.67%
SoloThreadMgr_2	28	10.74ms	13.21%
SoloThreadMgr_3	31	8.799ms	10.82%
SoloThreadMgr_1	22	6.747ms	8.29%
SoloThreadMgr_0	23	6.564ms	8.07%
SoloThreadMgr_5	32	6.264ms	7.70%
SoloThreadMgr_4	39	5.791ms	7.12%

- 2 RT tasks and 6 FAIR tasks have a similar load

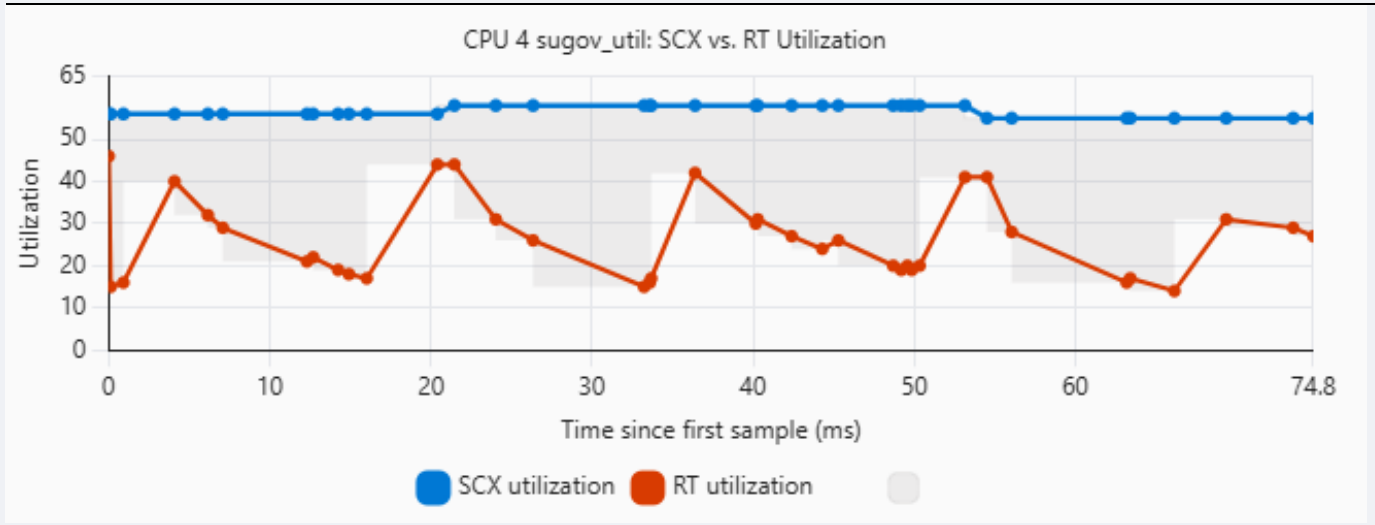
One sched-ext scheduler across RT + FAIR can make better use of all three CPUs.

How the schedutil governor can benefit from RT within sched-ext

TikTok livestream scenario



- 2 RT tasks run periodically in the same frame.
- 6 FAIR tasks run periodically in the same frame.
- Cpufreq bumps up at very end, when the scenario is about to finish.



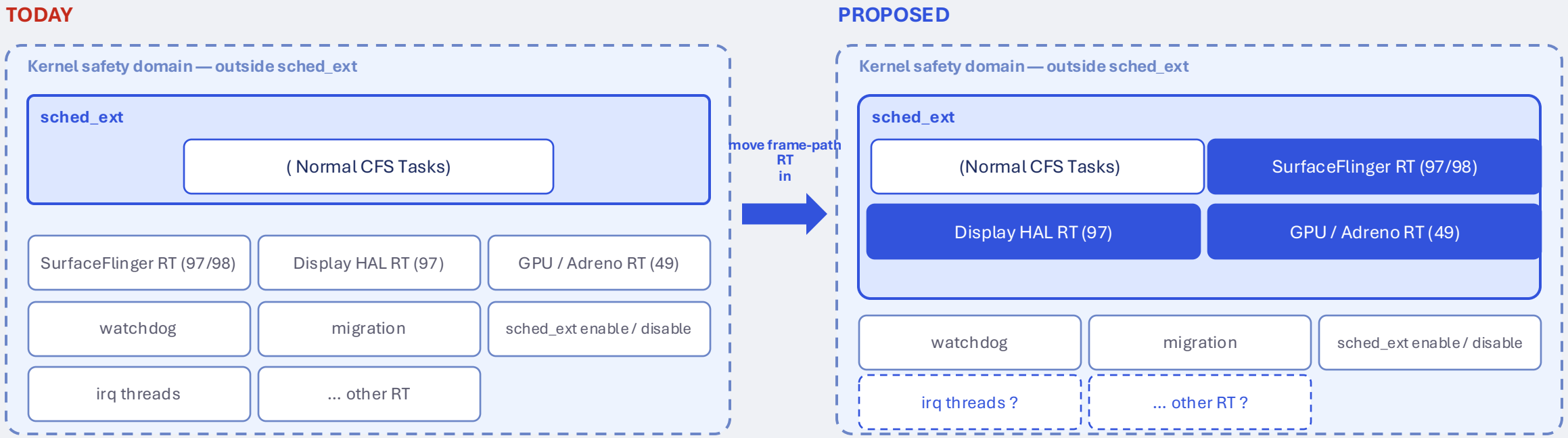
- RT utilization shows larger fluctuations and aligns with cpufreq bump point.

Chart: SCX utilization holds steady; RT swings ~15–45%

- **An aligned frame-window based utilization model is easier to provide from sched-ext.**
- **The sched-ext can privode a more stable sched_util input to governor.**

Where does the sched_ext control boundary sit?

Today every RT thread – frame path and system safety alike – runs in the native RT sched class, outside sched_ext



Special RT tasks

Should all RT tasks with priority > 50 be allowed to move into sched_ext, or must some of them stay in the native RT class?

IRQ threads

Since IRQ threads run at priority 49, should they be included in sched_ext too, or should we keep them under rt_sched_class and honor preferred_rt_cpus_mask there?

Open for discussion

Is it safe to temporarily convert RT tasks to fair-class tasks while sched_ext is enabled, and then restore them to RT when sched_ext is disabled?
How can we move toward an upstream solution to avoid android vendor hook?

Thank you

